

# Linear Regression for Classification

Advanced Machine Learning

Sam Ogden

# DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

# Learning objectives

- Give categories their own model output, via one-hot labels and softmax
- Define cross-entropy loss for that output
- Show its gradient looks just like linear regression's

# Classification

# The Big Question

Predicting *which*, not *how much*

# What we have vs. what's different

## What we already have

- A dataset, a model, a loss
- A way to train it — all covered previously
- All of it assumed the answer is a single number

## What's different now

- The answer is a **category**
  - spam or not spam
  - cat or dog
  - which digit
- No natural “how far off” between categories, the way there is between numbers

# The Big Idea

**Some questions don't have a numeric answer — they have a category as the answer.**

# A concrete example we'll carry through

We'll sort pieces of fruit into **apple**, **banana**, or **orange**, from two features:

| weight (g) | color score (0=green, 1=red) | fruit  |
|------------|------------------------------|--------|
| 150        | 0.9                          | apple  |
| 120        | 0.2                          | banana |
| 180        | 0.7                          | orange |

# Why not just reuse linear regression directly?

# Categories aren't numbers

- We're used to fitting a line to numbers
- Apple, banana, orange aren't numbers — there's no natural “less than” or “far apart” between them
- Coding them as 0, 1, 2 invents structure that isn't really there

# The Big Idea

Each *individual* category is binary — it is, or it isn't.

# Let's just try it anyway

- Code the labels as integers: apple = 0, banana = 1, orange = 2
- Fit the usual linear model against those codes:

$$\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$$

- Predict a class by rounding  $\hat{y}$  to the nearest code

# Two problems with this

- **We invented an ordering** — this encoding says banana (1) sits *between* apple (0) and orange (2); relabel the fruits differently and the model's behavior changes
- **The output doesn't match the question** —  $\hat{y}$  can be anything (-1.3, 4.7, ...); rounding to the nearest code is a patch, not a fix, and a 1.99 vs. a 1.01 both round to “banana” without feeling equally confident

# What we actually need

- A way to write “which category” without implying order or distance → **one-hot encoding**
- A model output that’s bounded and behaves like a confidence → **softmax**

# One-hot encoding

# The Big Idea

**Give every category its own slot, and just mark which one is true.**

# Our fruit, one-hot

| fruit  | apple | banana | orange |
|--------|-------|--------|--------|
| apple  | 1     | 0      | 0      |
| banana | 0     | 1      | 0      |
| orange | 0     | 0      | 1      |

## Note

Compare to the integer coding from a few slides ago — apple, banana, and orange no longer have any numeric relationship to each other at all

# Written out

- $K$  categories  $\rightarrow$  a vector of length  $K$ , one slot per category
- Slot  $j$  is 1 if the example belongs to category  $j$ , 0 otherwise:

$$y_j = \begin{cases} 1 & \text{example is class } j \\ 0 & \text{otherwise} \end{cases}$$

- Exactly one slot is 1 — that’s the “hot” one — and its *position* carries all the information; nothing else in the vector does

# A “linear” model, per class

Linear scores — for now

# One score per class (recap)

- One shared model, not  $K$  separate ones —  $\mathbf{W}$  has one row per class,  $\mathbf{b}$  one entry per class:

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad o_j = \mathbf{w}_j \cdot \mathbf{x} + b_j$$

- $\mathbf{o}$  is just this layer's raw output (hence the letter) — same forward pass as multi-output regression, covered previously
- Each  $o_j$  is called a **logit**: the model's raw, unnormalized opinion about class  $j$  — it can be negative, has no upper bound, and the  $o_j$ 's don't sum to anything meaningful

## ! Important

$\mathbf{o}$  is not our prediction yet — one more function still needs to act on it

# Worked example: our fruit

Suppose, for one particular (weight, color) example, evaluating  $o_j = \mathbf{w}_j \cdot \mathbf{x} + b_j$  for each class gives:

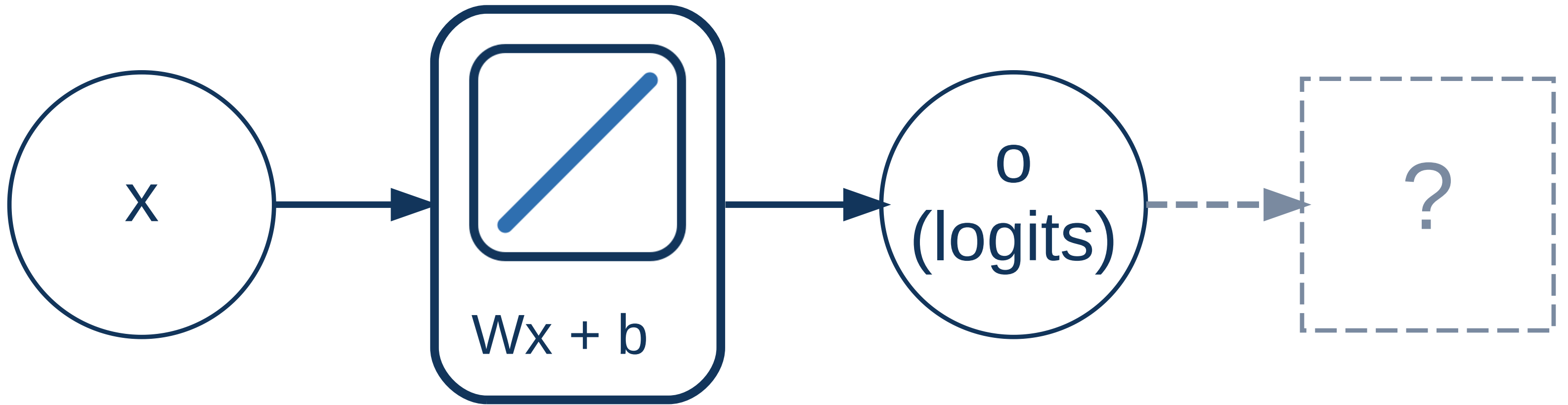
| apple | banana | orange |
|-------|--------|--------|
| -0.3  | 2.1    | 0.6    |

- Banana has the highest logit, so *right now* the model favors banana

 Warning

So the answer is just “banana,” done? That’s not actually what we do — why not?

# The picture so far



- $x \rightarrow$  linear layer  $\rightarrow o$  is the whole story so far
- Something still needs to turn  $o$  into an actual prediction — that's next

# Softmax

From scores to probabilities

# The Big Idea

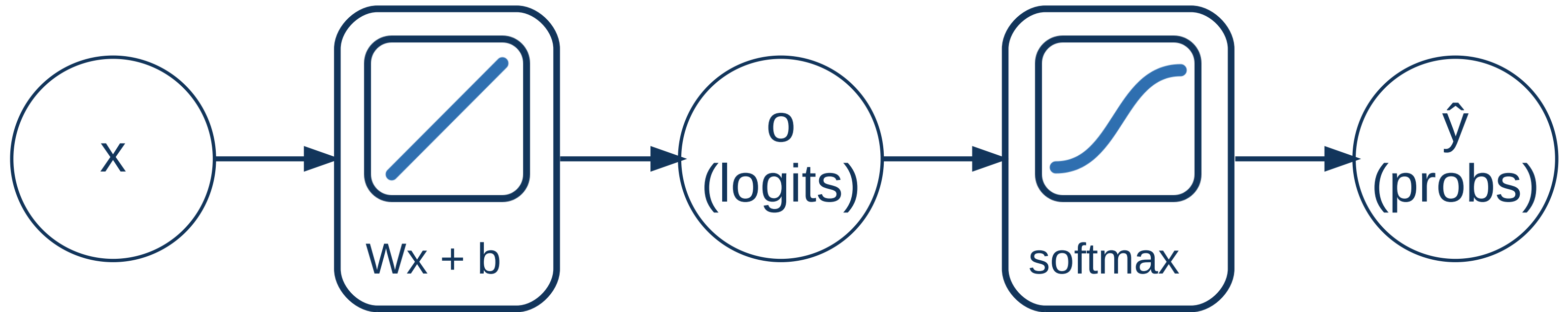
**We make each prediction *act* like a probability – without changing which class wins.**

# The softmax function

$$\hat{y}_j = \text{softmax}(\mathbf{o})_j = \frac{e^{o_j}}{\sum_{k=1}^K e^{o_k}}$$

- Takes in all  $K$  logits at once, returns one probability per class
- $\hat{\mathbf{y}}$  is now our model's actual prediction — a full probability distribution over the classes

# The full picture



- Two functions, applied one after the other: a **linear** layer, then **softmax**
- $o$  was never the answer — it's an intermediate value that softmax turns into  $\hat{y}$

# Breakdown: numerator and denominator

## Numerator

$$e^{o_j}$$

- Exponentiate *this* class's logit
- Always positive
- Same ranking as  $o_j$

## Denominator

$$\sum_{k=1}^K e^{o_k}$$

- The *same kind of term*
- Summed over every class
- Includes  $j$  itself

### Note

Divide one by the other: squeezes every class's share into  $[0, 1]$ , and all  $K$  shares add up to exactly 1

# So what does this actually do?

- It's not just squashing into  $[0, 1]$  — exponentiating also **magnifies** the gaps between logits
- Our banana/orange logits differed by only 1.5; their probabilities differ by a factor of  $e^{1.5} \approx 4.5$
- A modest lead in logits turns into a much larger lead in confidence

# Worked example: our fruit

Recall our logits: apple =  $-0.3$ , banana =  $2.1$ , orange =  $0.6$

|             | apple | banana | orange |
|-------------|-------|--------|--------|
| $e^{o_j}$   | 0.74  | 8.17   | 1.82   |
| $\hat{y}_j$ | 0.07  | 0.76   | 0.17   |

- Same winner as before (banana) — softmax never changes who wins
- But now we also know *how confident*: about 76%, not just “highest”

# The special case: two classes

- With only  $K = 2$  classes, softmax simplifies to:

$$\hat{y}_1 = \frac{1}{1 + e^{-(o_1 - o_2)}}$$

- That's the **sigmoid** function, applied to the *difference* between the two logits
- This special case has its own name — **logistic regression** — softmax is its generalization to more than two classes

# What softmax buys us

- **A real probability distribution:** non-negative, sums to 1, ranks classes the same way the logits did
- **A smooth function:** softmax is differentiable, unlike `argmax`
  - That matters a lot! To train with gradient descent, we need a loss we can actually take a gradient of

# Loss: Cross entropy

Loss function for multi-classification

# Why not just reuse MSE?

- MSE only knows “off by how much”; it can’t tell *unsure* apart from *confidently wrong*
- Being *confidently* wrong should cost a lot more than being unsure

# The Big Idea

**Penalize the model by how little probability it gave the correct answer.**

# Cross-entropy loss

$$L(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^K y_j \log \hat{y}_j$$

- $\mathbf{y}$ : the one-hot true label
- $\hat{\mathbf{y}}$ : the softmax output — our predicted probabilities
- One number, same as MSE was — small if we're right, big if we're wrong

# Only one term survives

- $y_j$  is 0 for every class except the true one, call it  $t$
- Every term in the sum vanishes except  $j = t$
- The whole sum collapses to just:

$$L = -\log \hat{y}_t$$

- Cross-entropy only ever looks at the probability the model gave the *correct* class

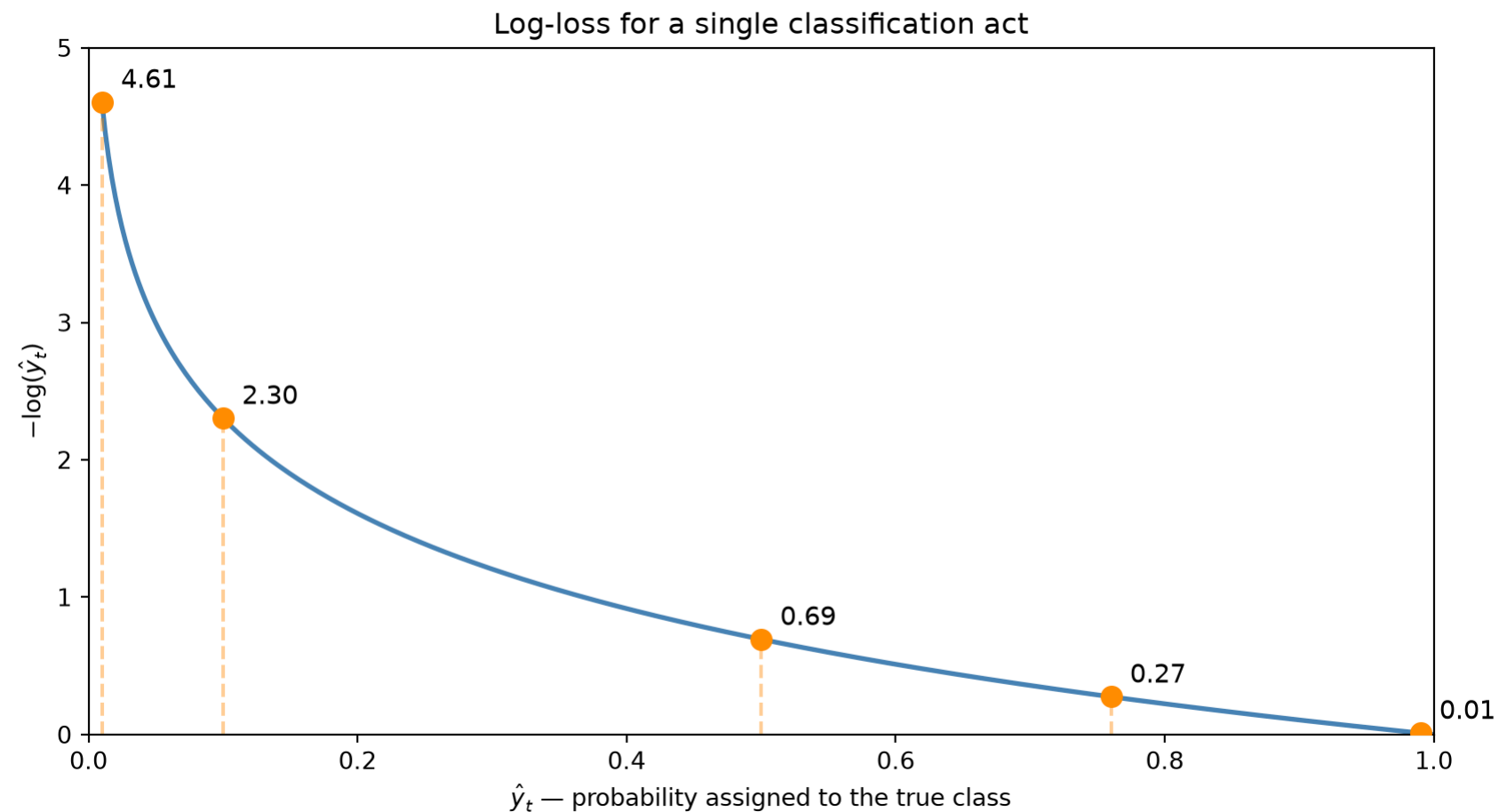
# Breakdown: why the negative log?

| $\hat{y}_t$ (true class probability) | $-\log \hat{y}_t$ |
|--------------------------------------|-------------------|
| 0.99                                 | 0.01              |
| 0.76                                 | 0.27              |
| 0.50                                 | 0.69              |
| 0.10                                 | 2.30              |
| 0.01                                 | 4.61              |

- Confident *and* correct ( $\hat{y}_t$  near 1)  $\rightarrow$  loss near 0
- Confident *and* wrong ( $\hat{y}_t$  near 0)  $\rightarrow$  loss blows up
- Being unsure costs a little; being confidently wrong costs a lot

# Log-loss, visualized

One classification act — one example, one true class — costs  $-\log \hat{y}_t$ :



- The curve is steep near 0, flat near 1 — the same asymmetry as the table, just visible now
- The five orange points are exactly the five rows from the table

# Worked example: our fruit

Recall our softmax output: apple = 0.07, banana = 0.76, orange = 0.17 — and the true label is banana

- $L = -\log(0.76) \approx 0.27$
- Had the model instead favored apple, with only 0.07 on banana:  
 $L = -\log(0.07) \approx 2.66$
- Same “wrong or right” call either way — cross-entropy is what tells them apart

# Differentiating the loss

Surprisingly easy calculus

# The Big Idea

**This scary-looking loss has a gradient just as simple as linear regression's.**

# Rewrite the loss and softmax first

Recall  $L = -\log(\hat{y}_t)$ , where  $\hat{y}_t = \frac{e^{o_t}}{\sum_k e^{o_k}}$  — substituting one into the other **fuses log-loss and softmax into a single expression**, before we differentiate anything:

$$-\log \frac{e^{o_t}}{\sum_{k=1}^K e^{o_k}} = -\log(e^{o_t}) + \log \sum_{k=1}^K e^{o_k} = -o_t + \log \sum_{k=1}^K e^{o_k}$$

- Step 1:  $\log(a/b) = \log a - \log b$
- Step 2:  $\log(e^{o_t}) = o_t$  — true because log here means the **natural log** (ln), not  $\log_{10}$
- Still just algebra — no derivative yet. We're rewriting  $L$  into a form that's easy to differentiate next

# Take the derivative

- Differentiate term by term with respect to  $o_j$ :

$$\frac{\partial L}{\partial o_j} = -y_j + \frac{e^{o_j}}{\sum_{k=1}^K e^{o_k}}$$

- The  $-o_t$  term only depends on  $o_j$  when  $j = t$  — its derivative is  $-1$  then, 0 otherwise, which is exactly  $-y_j$
- The second term is just softmax again,  $\hat{y}_j$  leaving us with:

$$\frac{\partial L}{\partial o_j} = \hat{y}_j - y_j$$

# Breakdown: the same shape as before

- Regression's gradient, covered previously:  $\frac{\partial L}{\partial w} = (\hat{y}_i - y_i) x_i$  — “prediction minus truth, times input”
- Same shape here, just once per class:

$$\frac{\partial L}{\partial \mathbf{w}_j} = (\hat{y}_j - y_j) \mathbf{x}, \quad \frac{\partial L}{\partial b_j} = \hat{y}_j - y_j$$

## ⚠ Important

All that softmax and log-loss machinery, and the gradient still just says “how wrong were we” — negative for the true class (push its logit up), positive for every other class (push them down)

# Putting it together: training a classifier

Same loop, new loss

# The Big Idea

**Nothing about the training loop changes  
— only the loss, and what “prediction”  
means.**

# What changed / what didn't

## Same as before

- The training loop itself — sample, gradient, step, repeat
- The learning rate  $\eta$
- $(\mathbf{W}, \mathbf{b})$  as the trainable parameters

## New this lecture

- Prediction: a probability vector, via softmax
- Loss: cross-entropy, not MSE
- Gradient:  $\hat{y}_j - y_j$  — same shape as before, just  $\hat{y}$  is now *computed* differently (via softmax, not a plain dot product)

# Summary

# The recipe

|                     |                                                                 |
|---------------------|-----------------------------------------------------------------|
| <b>Task</b>         | classify each example into one of $K$ categories                |
| <b>Example</b>      | apple / banana / orange, from weight + color                    |
| <b>Model output</b> | logits $\rightarrow$ softmax $\rightarrow$ a probability vector |
| <b>Loss</b>         | cross-entropy                                                   |

## Note

This recipe — categorical output, softmax, cross-entropy — is the template for classification problems in general, not just this one

## Tip

In lab: walk one example through a full forward pass (logits  $\rightarrow$  softmax  $\rightarrow$  cross-entropy) and backward pass (the gradient) **by hand**, with actual numbers